tt_um_santhosh_rvsoc: inputs and outputs, pin by pin

This page answers two questions: **what do I give the chip** and **what will I see come out**. It applies to the real Tiny Tapeout chip and to the PYNQ-Z2 FPGA version (right-hand column).

The chip has 26 signals: 8 inputs (`ui`), 8 outputs (`uo`), 8 bidirectional pins (`uio`, used for the memory bus), plus `clk` and `rst_n`.

---

1. What you must give the chip (inputs)

1.1 Clock and reset

Signal	What to give	Notes	PYNQ-Z2
`clk`	64 MHz square wave	Lower works too; the UART baud rate and timer scale with it (at 32 MHz you get 57600 baud unless firmware changes UART_DIV)	made from the 125 MHz oscillator by an MMCM
`rst_n`	Low = reset, high = run	Hold low for at least 8 clocks; raise it while `clk` is high	**BTN0** (hold = reset)

1.2 Memory: QSPI Pmod on the bidirectional pins (required)

The CPU has no internal program memory. It runs code from an external flash and keeps its data in external PSRAM, all on one shared 4-bit QSPI bus.

Pin	Direction after reset	Function
uio[0]	output	Flash CS# (active low)
uio[1]	in/out	QSPI SD0
uio[2]	in/out	QSPI SD1
uio[3]	output	QSPI SCK (= clk / 2, 32 MHz)
uio[4]	in/out	QSPI SD2
uio[5]	in/out	QSPI SD3
uio[6]	output	PSRAM A CS#
uio[7]	output	PSRAM B CS#

What has to be in the flash: the program image `firmware/build/rvsoc_fw.bin` at flash address 0. The flash must be in **continuous-read (XIP) mode** and both PSRAMs in **QPI mode** before reset is released; the TT demo board does this.

During reset all uio pins are inputs, and the chip reads its **QSPI read latency** from SD2:SD0 (uio[4], uio[2], uio[1]) = 0..5. Use 2 or 3 at 64 MHz on real hardware; the FPGA version uses 2.

On the FPGA the whole Pmod is emulated in block RAM, so nothing has to be connected and the firmware is built into the bitstream.

1.3 Dedicated inputs `ui[7:0]`

Pin	Name	What to give	What it does	PYNQ-Z2
ui[0]	IRQ0 / GPIO in0	button (low, pulse high)	Rising edge = interrupt 0; demo prints `IRQ0`	**BTN1**
ui[1]	IRQ1 / GPIO in1	button (low, pulse high)	Rising edge = interrupt 1; demo prints `IRQ1`	**BTN2**
ui[2]	MODE0 / GPIO in2	switch, read **at reset**	Mode select bit 0 (see table below)	**SW0**
ui[3]	MODE1 / GPIO in3	switch, read **at reset**	Mode select bit 1	**SW1**
ui[4]	FAST / GPIO in4	**keep low** on hardware	High = simulation mode (2 Mbaud, fast tick)	tied low
ui[5]	GPIO in5	anything	General-purpose input	**BTN3**
ui[6]	GPIO in6	anything	General-purpose input	Pmod JA pin 4
ui[7]	UART RX	serial data from your PC, 3.3 V, idle high	Receive, 115200 8N1	Pmod JA pin 2

Firmware mode (read once at reset from ui[3:2]):

ui[3]	ui[2]	Mode
0	0	**DEMO** (default): banner, self-test, LED chaser, UART echo, IRQ messages
0	1	**GPIO MIRROR**: uo[7:2] copies ui[7:2] continuously
1	x	same as DEMO

---

2. What you will see (outputs)

2.1 Dedicated outputs `uo[7:0]`

Pin	Name	What you see	PYNQ-Z2
uo[0]	UART TX	Serial text at 115200 8N1, idle high	Pmod JA pin 1 and JB pin 1
uo[1]	UART RTS	Low = ready to receive (active low)	Pmod JA pin 3 and JB pin 2
uo[2]	GPIO out2	LED chaser position 0	**LD0** and JB pin 3
uo[3]	GPIO out3	LED chaser position 1	**LD1** and JB pin 4
uo[4]	GPIO out4	LED chaser position 2	**LD2** and JB pin 7
uo[5]	GPIO out5	LED chaser position 3	**LD3** and JB pin 8
uo[6]	GPIO out6	LED chaser position 4	JB pin 9
uo[7]	GPIO out7	LED chaser position 5	JB pin 10

(Firmware can also hand uo[0]/uo[1] to GPIO through the GPIO_SEL register.)

2.2 UART output, DEMO mode (115200 8N1)

Right after reset (about 2 ms):

```
Hello, TT!
ISA PASS          <- every RV32EC / Zicsr / trap instruction checked
PSRAM PASS        <- words, halfwords and bytes written and read back on PSRAM A and B
TIMER PASS        <- timer compare interrupt fired 3 times, correct mcause
GPIO PASS         <- GPIO_OUT / GPIO_SEL write + read-back
SELFTEST PASS
```

Then, while it runs:

```
tick 10           <- every second (10 timer interrupts of 100 ms)
tick 20
IRQ0              <- whenever ui[0] goes 0 -> 1
IRQ1              <- whenever ui[1] goes 0 -> 1
<your text>       <- anything you type is echoed back (Enter gives a new line)
```

If a line says `FAIL n`, the number tells you which check failed (for ISA it is the test number in `firmware/isa_test.S`).

2.3 LEDs, DEMO mode

One LED lit at a time, moving uo[2] -> uo[7] -> uo[2] ..., one step every 100 ms (each step is one timer interrupt). On the PYNQ-Z2 you see LD0-LD3 light in turn, then a gap while the light is on uo[6]/uo[7] (Pmod JB).

2.4 GPIO MIRROR mode (reset with ui[3:2] = 01)

UART prints `Hello, TT!`, `GPIO PASS` and `MIRROR`. After that, uo[2..7] follow ui[2..7] about 1 microsecond later (for example, flip SW0 and LD0 follows).

2.5 PYNQ-Z2 status LEDs (FPGA only, no PC needed)

LED	Meaning
LD4 blue, blinking	FPGA is configured and the clock runs
LD4 green	the chip printed `Hello, TT!` (booted from emulated flash)
LD5 green	the chip printed `SELFTEST PASS`
LD5 red	the chip printed a `FAIL` line

---

3. Timing summary

Event	Time at 64 MHz
Reset low (minimum)	8 clocks = 125 ns
Reset released -> `Hello, TT!` complete	about 1.1 ms
Banner -> `SELFTEST PASS`	about 5 ms (mostly UART time)
Timer tick (demo)	100 ms (TIMER_CMP step 800000 x 8 clocks)
`tick N` line	every 1 s
UART bit time	8.69 us (556 clocks) = 115108 baud (-0.08 %)

---

4. Quick checklist

Real chip: flash `rvsoc_fw.bin` -> clock 64 MHz -> latency 2 on SD2:SD0 -> reset -> terminal at 115200 on uo[0]/ui[7] -> you should see `SELFTEST PASS` and the running LED.

PYNQ-Z2: program `fpga/build/rvsoc_pynq_z2.bit` -> press and release BTN0 -> LD4 turns green, then LD5 turns green, and LD0-LD3 start chasing. For the full console, connect a 3.3 V USB-UART: adapter RX to JA pin 1, adapter TX to JA pin 2, GND to JA pin 5 (or 11), 115200 8N1.